Comparing the 208-, 209-, and 196-state consistency machines

Alternative command interpreters and a reproducible reduction search

Research prepared for Vladimir Reshetnikov

13 November 2026

Abstract

The requested target is a binary, single-tape, blank-start Turing machine with fewer than 196 working states for which PA proves that nonhalting is equivalent to Con(PA). **This investigation does not establish such a machine.** Its smallest retained state count is 196. The deliverable is a comparison and research package, not a claim to a new Busy Beaver state-count record.

We compare every relevant incoming archive at a pinned ProveIt snapshot, the published 209-state machine, and its 196-state successor. The nine designated tables in five principal 208-state submissions form two rooted transition-graph classes; a further packet supplies eighteen locally proved variants. We reconstruct the 196-state AX decoder exactly, identify why the old 208-state fold is not an additional saving on that decoder, and give two one-row changes to the 196-state interpreter that reduce its source instruction count. The stronger change returns the modus-ponens consequent in a different register and has an explicit complete command compiler. Neither change lowers the physical state count. A paper-level syntactic argument supplies the missing direction of the PA consistency equivalence, with its unimplemented formalization obligations stated explicitly. Failed physical layouts and the finite scopes of unsuccessful searches are retained as reproducible evidence.

Certification boundary. The upstream Lean sources are included unchanged. Their publication/table checks were run successfully; the full Lean build was not rerun in this environment. New command proofs are mathematical arguments, accompanied by bounded executable checks. There is no new machine-checked PA biconditional and no certified machine with at most 195 working states in this archive.

Contents

1 Target, conventions, and provenance **3**
1.1 Frozen sources . . . 3
1.2 Evidence levels . . . 3

2 Census of the incoming 208-state designs **4**
2.1 Two principal machines, with additional helper variants . . . 4
2.2 The exact conditional-history fold . . . 5
2.3 Why the old saving does not extend PA196 directly . . . 6

3 The 209-state and 196-state published designs **6**
3.1 Logical and coding changes . . . 6
3.2 Physical sharing . . . 7
3.3 What the published formal endpoints assert . . . 7

4 Conservative changes to the command interpreter **7**

5 A checked family of physical register permutations **8**

6 The exact numerical calculus behind the comparison **8**

7 The new literal 196-state variants and their limits **9**
7.1 A source simplification which increases sharing cost . . . 10
7.2 A rejected physical realization of the simpler Shift . . . 10

8 Command reductions and a complete proof compiler **10**
8.1 Two independent simplifications . . . 11
8.2 Unchanged auxiliary words . . . 11
8.3 Compiling modus ponens with a zero below the premises . . . 12
8.4 A transport proof for the Shift change alone . . . 12
8.5 A weaker MP option . . . 13

9 The reverse implication inside Peano arithmetic **13**
9.1 A syntactic interpretation into PA . . . 13
9.2 Uniform finite comprehension is a proof schema . . . 14
9.3 Command safety under consistency . . . 15
9.4 From commands to the literal tape machine . . . 15
9.5 What ConciseIPRA contributes . . . 16

10 Reduction searches and the meaning of their failures **16**
10.1 Finite-window local folds . . . 17
10.2 Grammar, coding, and physical-price searches . . . 17
10.3 Scanner and axiom-removal proposals . . . 18

11 Reproduction, artifact map, and remaining obligation **18**

1 Target, conventions, and provenance

For a literal machine M, let H_M be the arithmetical sentence asserting the existence of a finite halting trace from its specified blank initial configuration. The intended result is

$$|Q| \leq 195, \qquad \mathrm{PA} \vdash (\neg H_M \leftrightarrow \mathrm{Con(PA)}).$$

The tape is infinite in both directions, its alphabet is $\{0, 1\}$, the blank symbol is 0, and each transition writes a symbol and moves one cell left or right. A separate terminal state has no outgoing rows and is not counted among the working states. In particular, the published 196-state machine has states $0, \ldots, 195$ and halt label 196. Its initial state counts as a working state. Renaming the terminal state, omitting the initializer from the count, or changing the alphabet would not meet the requested target under these conventions.

1.1 Frozen sources

All comparisons refer to the following exact snapshots, rather than to an unversioned interpretation of `main`:

PA196.
James Chang's BusyBeaver repository, `407cdc356f6b586635e71ac92be394255f4bfea4` [1].

PA209.
The requested historical BusyBeaver snapshot, `69d98241a721ead20b4f0b17df9062950b40fbfe` [2].

Incoming 208-state material.
ProveIt, `239b7a406e24405d464edc526e09d402abce5bc5`, directory `docs/incoming` [3].

ConciseIPRA.
The directory `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA` at the same ProveIt snapshot [4].

The eleven incoming ZIP files were checked against their Git blob SHA-1 identities. Their SHA-256 digests, sizes, source URLs, member counts and literal-table inventory are recorded in `sources/incoming208/incoming_comparison_manifest.json`. The original ZIP files are not duplicated in this package; the compact proof and table material needed by the included replay is retained.

The two published literal-table SHA-256 digests are:

PA196.
`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`

PA209.
`0610d978675320f0b4b1ef34619dcd3daa5912632d7037e200dfcc70954fea51`

1.2 Evidence levels

Four different assertions must remain separate. A literal table check binds a file to a state count and a hash. A local equivalence proof shows that a transformation preserves computation under proved entry invariants. A source or command proof explains the computation at a higher level. Finally, an internal PA proof connects the exact trace sentence to a syntactic

consistency predicate. Passing examples at one level does not discharge the obligations at the others.

The positive finite local checks for the eighteen 208-state helper variants quantify over all relevant two-cell patterns and leave the exterior tape opaque. They are finite proof checks. By contrast, the new 196-state primitive-frame checks use concrete register values and both decrement outcomes; they are bounded falsification tests. The manuscript supplies the accompanying command arguments but does not replace a missing uniform tape formalization by those examples.

2 Census of the incoming 208-state designs

2.1 Two principal machines, with additional helper variants

The five principal construction packets contain nine designated output tables. Canonical breadth-first numbering from the specified initial state, visiting the read-zero edge before the read-one edge and preserving every write and move, puts those nine tables in exactly two classes. The comparison checks full rooted graph isomorphism, not just row counts or sample executions. The two classes have the following canonical hashes under the serialization in `compare_current_inputs.py`:

Class A.
`a09b3b5d5d0ea5300d3bbf94b75987e573f4434ef685aebc217bd5516ae8ca7d`

Class B.
`2aba76c0ac54cddd76c92acb14a9e3e7cdcb12bc1bf0245024d5bb1f0cb6cbf5`

The eighteen helper variants in `pa208_research_2026-10-29.zip` are eighteen further graph classes, nine for each base class. Their same-count local equivalences were freshly checked. They are not sub-208 constructions. Across all extracted material there are 545 literal TM files, including 316 with 208 working states and 142 graph classes among those 208-state files. Those large totals include rejected and exploratory machines; they are not counts of accepted constructions.

Archive stem (all dated 2026-10-29 except the final named cover)	Contribution and status
`U208_ConPA`	Principal class A. A different native labeling of the same conditional-history reduction.
`U208_ConPA_Conditional_Context`	Principal A and alternative B. Complete local proof of the 172/211 fold, using helper 217 or 214.
`U208_ConPA_Entry_Context`	Principal A and alternative B; final dense tables are byte-identical to the corresponding Conditional Context tables. The proof organization differs.
`U208_ConPA_History`	A and B in another native labeling; presents the finite-history invariant.
`u208_research`	Principal B and alternative A. Retains full symbolic runtime receipts, 5,084 contracts for each final table.
`pa208_research`	Retains A/B and adds eighteen proved 208-state helper variants. Its additional source rotations and emitter changes do not provide an accepted smaller machine.
`Below_U208_Research`	Research on boundary projections, head offsets and source pruning. No accepted smaller machine; search scopes are finite.

Archive stem	Contribution and status
`ConPA_below208_research`	Research on source/output role collapse, parameter permutations and local merges. Includes a two-element countermodel to one proposed collapse.
`pa_consistency_research`	A 3,909-to-3,904 source-instruction specialization and context research. No reduction of the literal working-state count; a 207-state rotation is rejected.
`u208_comparison_research`	Deduplicates the main outputs and records bounded searches and a rejected 207-state return-path counterexample.
`pa_unary_power_cover.zip`	Unary theory and abstract numerical algorithm, with no literal tape machine. Its dyadic theorem stream is distinct from the native Cantor-coded source interpreter.

The common ancestry is operationally useful but does not make different folds additive. The incoming U209 families descend from the documented U212 construction and combine deletions of native roles 180, 181 and 218. Their retained arithmetic entry at native row 171/0 is repaired from target 7 to target 205. The larger nested lineage includes U213, U214, U218 and U222 material. The separately published PA209 package uses the same register-engine/calculus family with its own source layout and formal proof package. Native state labels cannot be identified across these files without the supplied maps.

2.2 The exact conditional-history fold

Use the native labels of the Conditional Context packet, in which halt is 254. Its class-A reduction deletes state 172, redirects

$$116/0:\ 0R172 \longmapsto 0R211, \qquad 211/0:\ 0R157 \longmapsto 0L217.$$

The deleted rows are $172/0 : 0L67$ and $172/1 : 1R184$. The new state 211 represents both old roles. Their zero branches remain distinguishable because of an invariant of the tape to the left of the head. Writing t_j for the bit at offset j, the relevant conditions are

$$\begin{array}{r|l} 116 & t_{-1} = 1 \\ 172 & t_{-2} = 1 \ \wedge\ t_{-1} = 0 \\ 217 & t_{-1} = 1 \\ 163 & t_{-1} = 1 \ \vee\ t_{-2} = 1 \\ 211 & t_0 = 1 \ \vee\ t_{-1} = 1. \end{array}$$

The full incoming-edge census proves these conditions by induction from state 0 on every initial tape. In particular, the condition on 211 is conditional on its scanned symbol; asserting an unconditional left-hand one would be false.

The replacement blocks simulate one original transition in the following numbers of steps:

Original role/read	172/0	172/1	211/0	211/1
Replacement length	7	1	3	1

They restore exactly the original tape and head endpoint, with the state identification applied. If $N_{q,b}(n)$ counts occurrences of row (q, b) among the first n parent transitions, the simulation clock is

$$\tau(n) = n + 6N_{172,0}(n) + 2N_{211,0}(n), \qquad n \le \tau(n) \le 7n.$$

Thus the local proof establishes halting equivalence, including blank-start equivalence, and is arithmetizable. Multiple presentations of that same fold do not supply multiple state savings.

2.3 Why the old saving does not extend PA196 directly

The old roles 172 and 211 are absent from the new AX body. PA196 uses six different predecessor-bit folds in a new 68-node macro grammar. The independent compiler reconstructs its unfolded 202-state table and then reapplies those six folds; the result equals all 392 supplied rows, after the published dense/native renaming. Consequently, simply applying the 208-state patch to PA196 is not a defined transformation. The useful transfer is the method—recovering control information from proved tape context—rather than the old numerical labels.

3 The 209-state and 196-state published designs

The comparison uses the PA209 commit `69d98241a721ead20b4f0b17df9062950b40fbfe` and the PA196 commit `407cdc356f6b586635e71ac92be394255f4bfea4` of https://github.com/jamesyc/BusyBeaver. The latter is two commits after the former; commit `64ff1b3` replaces the proof package and `407cdc3` polishes its paper.

Quantity	PA209	PA196
Working states	209	196
Defined binary transition rows	418	392
Source instruction sites	3948	3812
Distinct internal source-tree nodes	173	159
Maximum instruction-path length	33	38
Startup steps to the candidate-2 frame	58210	58154
AX macro tokens under the common tokenization	210	201
AX internal grammar nodes under that tokenization	83	68

The last two rows are independently counted from the published Lean source trees. A token is one of push, push-and-copy of one of the three inputs, the Cantor constructor, increment, the parameter readers, or the final guard/control macros. The distinct-tree count includes every reachable internal node, not just nodes with names in the generated source.

PA209 uses sparse labels $0, \dots, 213$ excluding $50, 122, 150, 157, 205$, with halt label 215. PA196 uses consecutive working labels $0, \dots, 195$ and halt label 196. State renaming alone therefore obscures the useful structural comparison.

3.1 Logical and coding changes

Writing p for Cantor pairing, PA209 uses $4p(a, b) + r$ for all four constructors, with residues $0, 1, 2, 3$ for implication, membership, universal quantification, and equality. PA196 retains the implication, membership, and equality constructors but uses $U(a, B) = 4p(B, a) + 2$. This transposition makes repeated quantifier construction share more of the register-program grammar.

The decisive logical change replaces the seventh primitive-equality head by guarded equality vacuity $E(a, b) \to U(c, E(a, b))$. PA196 adds that clause to the separation gate, under its existing freshness guard $c \neq a, b$. Reverse membership rewriting is recovered using pointed separation; the interpretation of arithmetic then uses equality defined by matching both incoming and outgoing membership neighborhoods. It does not need primitive equality symmetry and transitivity as independent axioms.

The remaining six heads are reordered, two pairs of implication antecedents are exchanged, and the equality witness is mirrored. These changes improve repeated-subtree sharing while

preserving the required interpreted equality facts. The 196-state machine is therefore not simply the old table with thirteen duplicate rows removed.

3.2 Physical sharing

PA196 has 36 engine states, 13 leaf-routing states, and 147 decision states for its 159 distinct source-tree nodes. Six pairs of AX grammar nodes share controls using the bit immediately preceding the current path bit. For a conflicting branch, the shared control steps left, reads this saved choice, and uses existing controls to step back right into the correct continuation. These six reductions do not introduce additional helper states. The other existing control folds remain in the unchanged outer framework. Two obsolete outside-body distinction states of PA209 are removed.

An independent compiler in `compile_ax.py` reconstructs the unfolded 202-state PA196 decoder and reproduces the supplied 196-state transition table exactly after the published six AX folds. Thus its state-count model is checked against the literal machine.

3.3 What the published formal endpoints assert

Both packages explicitly prove an internal implication

$$\mathrm{PA} \vdash \neg H \to \mathrm{Con(PA)}$$

and an external consistency-to-nonhalting theorem, together with two-sided independence under the stated metatheoretic assumptions. Their README files and theorem tables do not advertise a checked internal converse $\mathrm{PA} \vdash \mathrm{Con(PA)} \to \neg H$. That converse requires an arithmetized syntactic soundness compiler; it must not be inferred merely from the external theorem.

4 Conservative changes to the command interpreter

Two independently checked program changes produce alternative 196-state tables. They can be combined with further decoder optimizations, but by themselves do not satisfy the strict target.

First, a successful MP command need not push a zero onto its stack. Changing native row $(164, 1)$ from target 98 to target 114 (dense row $(115, 1)$ from target 86 to target 100) removes the 19-site push block. The new source has 3793 sites. Resolving every remaining source instruction gives exactly the old control-flow graph with that block deleted and addresses shifted. All concrete primitive instruction contracts checked at clean frames, including both outcomes of each decrement, pass. At the command level MP success becomes $(b, w, b, 0)$ rather than $(b, p(0, w), b, 0)$. A refutation compiler can insert zero below the two premises using the old discard-and-push words before invoking the otherwise unchanged MP compilation.

Second, a successful MP command may destructively move its consequent from register D into visible register H. Native row $(164, 1)$ then targets 130 (dense target 108), and the source replaces the push-and-copy subtree by the already present three-instruction $D \to H$ loop. The resulting source has 3789 sites. All 4928 concrete primitive instruction cases pass. MP success is $(0, w, 0, b)$. With a zero inserted below the premises, the word consisting of two Shifts, MP(b), and one Shift returns $(b, w, 0, 0)$, or halts when $b = 1$. This supplies a particularly simple adaptation of the refutation compiler.

These concrete runs are falsification checks, not a substitute for a uniform proof over all natural register values. The supplied source and row changes make the required uniform proof local: the standard unary engine is unchanged, and the replacement loop has the elementary invariant that $D + H$ is constant until $D = 0$. For both changed tables, a separate exact finite computation confirms the complete original startup configuration after 58154 steps: candidate two, gap 48, and the physical file $[0, 0, 0, 0, 0, 0, 2, 0, 0]$ followed by 26 ones.

5 A checked family of physical register permutations

The logical registers are physically ordered by the permutation $[6, 2, 0, 4, 3, 7, 8, 5, 1]$. Retargeting the eleven non-engine rows which enter the fixed nine-state register-skip chain permutes these physical locations. Keeping the candidate counter at physical position six gives 28 elementary transpositions of the other eight positions. All 28 tables pass all 4958 primitive instruction cases in the clean-frame test profile, for a total of 138824 checked runs. None yielded a further state fold in the common-row finite-window search at radius four. This is a negative result for the specified finite search, not a lower bound on state complexity. The same 28 transpositions were also combined with the destructive MP variant and screened at radius four; no further fold was found. Those combined-table screens are not additional uniform simulation proofs.

Changing the physical order can also change bootstrap behavior. For the transposition of physical positions zero and one, an exact blank-tape run reaches candidate two after 43248 steps with gap 41 and 28 registers. That gap is below the published uniform bound. After 74404 steps it reaches a fully checked clean candidate-three frame with gap 53 and 41 registers; every other logical register is zero and the 32 spare registers are one. This later frame meets the published gap bound. One should therefore record the actual new startup frame, rather than expect the old 58154-step trace to survive a register permutation unchanged. The source scheduler's completeness argument can start at candidate three, since the compiled universal halting word has candidate index at least three.

6 The exact numerical calculus behind the comparison

The following definitions make the logical comparison explicit. They are the constructors in `PA196/Logic/Basic.lean` of the pinned source. For $a, b \in \mathbb{N}$, put

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \quad I(a,b) = 4p(a,b), \quad M(a,b) = 4p(a,b) + 1,$$

$$U(a,B) = 4p(B,a) + 2, \qquad E(a,b) = 4p(a,b) + 3.$$

Here $M(a,b)$ denotes the formula whose interpretation says that x_b is a member of x_a, and $U(a,B)$ binds variable x_a in the formula with code B. Define coded negation, conjunction and existence by

$$N(B) = I(B,1), \quad B \wedge C = N(I(B,N(C))), \quad \exists x_a B = N(U(a,N(B))).$$

Code 0 decodes as truth; code 1 decodes as $x_0 \in x_0$ and is the refutation target. It is not a primitive falsity symbol. The syntactic translation later in the report explicitly proves its falsity in PA.

In the following displays, an arrow abbreviates the numerical constructor I, so a displayed formula specifies an actual natural-number code. PA196's six unrestricted heads are

$$\begin{aligned}
&(\exists x_D\, U(D,H)) \to H,\\
&U(D,U(A,H)) \to U(A,U(D,H)),\\
&M(H,D) \to (E(H,A) \to M(A,D)),\\
&E(H,D) \to (M(A,H) \to M(A,D)),\\
&\exists x_D\, E(H,D),\\
&U(D,A) \to \big(U(D,A \to H) \to U(D,H)\big).
\end{aligned}$$

The restricted propositional head is

$$\rho_8(a,b,c,u) = ((c \to u) \to a) \to ((a \to c) \to (b \to c)), \qquad u \le a.$$

The numerical restriction matters to the available derivations even though every unrestricted instance is classically valid under the intended interpretation.

For $c \neq a, b$, define the gate from

$$\begin{aligned}
&\Delta = E(a,b) \to U(c,E(a,b)), && V = M(a,b) \to U(c,M(a,b)),\\
&P = c \to M(c,b), && S = U\big(b,(M(a,b) \to P) \wedge (M(c,b) \to c)\big),\\
&Q = U(c,S \to N(P)), && F(a,b,c) = N\big(\Delta \to (V \to Q)\big).
\end{aligned}$$

If the guard fails, the operational gate returns 0. The number c plays two roles in this definition: it is the matrix formula code and the fresh variable label used for the separation witness. The decoder bound that all variable labels occurring in code c are below c is therefore an essential part of the soundness proof.

PA209 uses the untransposed quantifier code $4p(a,B)+2$, a gate $N(V \to Q)$, and an additional equality head

$$\rho_7(a,b,c) = E(c,a) \to \big(E(c,b) \to U(M(a,b),E(a,b))\big).$$

PA196 removes this seventh head and adds Δ to the guarded gate. Thus the reduction is a change of a precisely specified calculus and its interpretation, followed by a better compiler layout. It cannot be justified by deleting the old equality-head emitter without proving the replacement equality facts.

7 The new literal 196-state variants and their limits

Both experimental tables keep dense state set $0, \ldots, 195$, start 0, halt 196, and all rows except $(115, 1)$ unchanged. Their exact patches are

Variant	Original row	New row
Omit successful-MP zero push	115 1 1 R 86	115 1 1 R 100
Return successful-MP result in H	115 1 1 R 86	115 1 1 R 108

All three tables reach exactly the same full tape configuration after 58,154 transitions: state 75, absolute head position -2, and the candidate-2 source frame. The changed row is not executed in this prefix. This is an exact finite startup check, not evidence of nonhalting by long simulation.

The file `experimental_mp_without_zero_push_196.tm` has SHA-256

`3d7f91e548f8a21ed4d6eecdc1f1299d960b612912bf9a6bf7b74870c5c9f11a`.

The file `experimental_mp_result_in_h_196.tm` has SHA-256

`893f08672746ea1c281fda7de4ecc7c8a75fa1f896f2255c9c00e345c37aaaca`.

These are explicitly labeled experimental because the altered end-to-end tape and PA theorems have not been instantiated and checked in Lean. The original published 196-state table remains the certified baseline.

For the stronger variant, the changed source contains 3,789 primitive sites, and all 4,928 nonhalting instruction cases in the concrete frame test pass. The tested profile has a fixed clean register frame; every decrement is tested at zero and at one. The uniform replacement loop has the invariant $D + H = b$, decreases D by one on each iteration, and takes $3b + 1$ primitive steps. A full tape proof must connect that source loop and every changed source address to the unchanged unary engine for arbitrary admissible frames. The frame tests are helpful evidence for that local task, but do not themselves quantify over arbitrary values.

7.1 A source simplification which increases sharing cost

A corrected three-category abstract interpretation, initialized at source site 25 with logical registers $(2, 0, 0, 0, 0, 0, 0, 0, 0)$, proves that register 6 is zero at site 7. Deleting the associated two primitive sites preserves the remaining source control-flow graph. Yet the original comparison subtrees are shared with other, unrestricted comparisons. Specializing only this occurrence increases the number of distinct internal source nodes from 159 to 161. Patching its shared physical decoder globally would change other command comparisons. This is a concrete example of why a smaller source instruction list need not mean fewer tape states.

7.2 A rejected physical realization of the simpler Shift

The command-level H-only Shift described below is valid. One attempted source reassociation to realize it with existing decoders is not. The attempt changes the unique outer continuation path from `101011` to `1010011`. The existing continuation router erases two terminal ones and crosses one zero before toggling a higher bit; the new path requires crossing two zeros. Its apparent reuse of the old router returns to path `1011` rather than the required `100000`.

The resulting `REJECTED_shift_layout.tm`, and the version combined with the new MP branch, both halt from the blank tape in exactly 43,563 transitions. They are counterexamples to that compilation attempt, not candidate consistency machines. The archive includes the bad tables, the failing frame, and a reproducible blank-run check. An exhaustive 784-action attempt to repair one existing routing row found no repair within that narrowly specified family. No impossibility claim follows.

8 Command reductions and a complete proof compiler

Write a command store as (t, w, d, h), where w is a Cantor-coded stack. For clarity, $a :: w$ denotes one stack item, including the zero-tail convention $0 :: 0 = 0$. Retain the axiom command unchanged. Its generated stack has the form

$$L_0 :: L_1 :: \cdots :: L_7 :: t :: w,$$

where $L_0, \dots, L_5$ are the six heads in reverse push order, L_6 is the restricted propositional head, and $L_7 = \mathsf{U}(d, h)$. Every L_i differs from code 1: the heads and propositional axiom are implication codes, and the last item has quantifier tag 2. The axiom command clears d and h.

8.1 Two independent simplifications

The original Shift halts when $t + h = 1$ and otherwise maps

$$(t, a :: w, d, h) \longmapsto (h, w, d, a).$$

The simpler Shift tests only $h = 1$, with the same nonhalting update. Its soundness invariant needs only a proof of the formula in h.

The original successful MP command, with $b = d + x$ and $\mathsf{I}(t, b) = h$, returns $(b, 0 :: w, b, 0)$. A stronger simplification returns the consequent directly in the other live formula register:

$$(t, w, d, h) \overset{\mathsf{MP}(x)}{\longmapsto} \begin{cases} (0, w, 0, d + x), & \mathsf{I}(t, d + x) = h, \\ (0, w, d + x, 0), & \text{otherwise.} \end{cases} \tag{1}$$

This removes the successful branch's zero push and its copying of the consequent back to t while retaining d. The failed branch is unchanged. If both live formula registers contain derivable formulas, the successful branch puts a derivable consequent into h and truth into t; the failed branch puts truth in both. Hence the same syntactic proof invariant applies. The two simplifications can be used together.

At the register-program level, the original Shift preparation first transfers T to the scratch register X, then transfers H to both T and X, then transfers X to H. Its effect is $(T, H, X) = (h, t + h, 0)$. Replace only the first transfer by the existing clear-T loop. The new effect is $(T, H, X) = (h, h, 0)$, so the unchanged one-test guard now tests the incoming h alone. Counting source instructions, the three loops take respectively $2t + 1$, $4h + 1$, and $3h + 1$ steps, a total of $2t + 7h + 3$. The guard takes eight steps when $h = 1$ and $2h + 7$ when $h \neq 1$; hence the halting branch takes $2t + 18$ steps and the nonhalting branch reaches stack decoding in $2t + 9h + 10$ steps. These counts concern source instructions and require the corresponding source-address changes; they are not tape-step counts.

For (1), after a successful comparison the live registers satisfy $T = H = X = 0$, $D = b$. The existing destructive-transfer loop from D to H has $3b + 1$ source steps and leaves $(T, H, D, X) = (0, b, 0, 0)$. It is therefore sufficient in place of the zero-push and the preserving transfer from D to T.

8.2 Unchanged auxiliary words

Use the original words

$$\mathsf{PUSH} = \mathsf{AX}(0, 0, 0)\ \mathsf{SHIFT}^8\ \mathsf{MP}(0),$$
$$\mathsf{DROP} = \mathsf{MP}(1)\ \mathsf{AX}(1, 0, 0)\ \mathsf{SHIFT}^9\ \mathsf{MP}(0).$$

Their exact contracts remain

$$\mathsf{PUSH}(t, w, d, h) = (0, t :: w, 0, 0), \qquad \mathsf{DROP}(t, w, 0, 0) = (0, w, 0, 0).$$

Every MP instruction in these words takes its failed branch. For PUSH, after eight shifts the value in h is a quantifier code, whereas an implication code is divisible by four, so equality cannot hold. For DROP, the first test compares the positive code $\mathsf{I}(t, 1)$ with zero; the final

test compares $\mathsf{I}(\mathsf{U}(1,0),0)$ with zero. The offset word $\mathsf{MP}(u)$ for $u>0$, applied to $(0,w,0,0)$, also fails and establishes $(0,w,u,0)$. Thus these contracts do not depend on the old successful MP branch.

The eight- and nine-shift identities following AX are unchanged even with the new Shift. The former exposes L_6 and L_7 while retaining the saved top in the stack; the latter gives

$$\mathsf{AX}(c,x,y)\ \mathsf{SHIFT}^9(t,w,d,h)=(\mathsf{U}(d,h),w,0,t). \tag{2}$$

All nine tests examine either zero or one of the non-one generated heads.

The original word $\mathsf{POP}=\mathsf{AX}(0,0,0)\mathsf{SHIFT}^{10}$ now has the contract

$$\mathsf{POP}(t,a::w,d,h)=\begin{cases}\text{halt}, & t=1,\\ (t,w,0,a), & t\neq 1.\end{cases}$$

This distinction matters: POP is no longer unconditionally total under the h-only halt test. It does not affect the axiom selectors because their selected heads and gates differ from 1. The original generalization word is unchanged: it uses PUSH, an offset, one Shift, then AX and nine Shifts, and returns $(\mathsf{U}(i,a),w,0,0)$ from $(a,w,0,0)$.

8.3 Compiling modus ponens with a zero below the premises

Suppose p proves A of code a and q proves $A\to B$ of code $\mathsf{I}(a,b)$. Define the new compiler recursively by retaining the original axiom selectors and generalization clause, and replacing its MP clause with

$$\mathsf{DROP}\ \mathsf{PUSH}\ C(q)\ \mathsf{PUSH}\ C(p)\ \mathsf{PUSH}\ \mathsf{SHIFT}^2\ \mathsf{MP}(b)\ \mathsf{SHIFT}. \tag{3}$$

The initial DROP and PUSH insert a zero below both premises. Starting with $(t,w,0,0)$, unless a recursive call already halts, the states at the significant boundaries are

after the first DROP and PUSH	$(0,0::w,0,0)$
after $C(q)$ and PUSH	$(0,\mathsf{I}(a,b)::0::w,0,0)$
after $C(p)$ and PUSH	$(0,a::\mathsf{I}(a,b)::0::w,0,0)$
after two Shifts, if $a\neq 1$	$(a,0::w,0,\mathsf{I}(a,b))$
after MP(b)	$(0,0::w,0,b)$
after the final Shift, if $b\neq 1$	$(b,w,0,0)$.

The second Shift halts if $a=1$, and the last Shift halts if $b=1$. Both conclusions hold for either halt test: at these tests $t=0$. The compiler therefore preserves an arbitrary incoming stack and returns the derived formula with cleared caches, or halts earlier.

Induction on the finite derivation proves completeness. For a derivation of code 1, no axiom or generalization can be its final constructor, because their conclusion codes differ from 1. Its last constructor is MP, whose final Shift halts. Thus the compiled word itself halts. Prepending one AX resets arbitrary initial caches, exactly as in the upstream compiler. The construction is primitive recursive and uses only fixed words and recursive traversal of a finite derivation; its correctness proof is an arithmetical induction available in PA.

8.4 A transport proof for the Shift change alone

There is also a direct way to preserve the original refutation compiler when only Shift changes. Given an old halting word W and its initial store, evaluate the finite word to locate its first

halting Shift. Let U be the preceding prefix and (t, w, d, h) its store. Evaluation is bounded by the word length, so this transformation is primitive recursive. The new execution of U either already halts, or reaches that identical store: every nonhalting update is unchanged. If $h = 1$, append one Shift. Otherwise the old condition $t + h = 1$ forces $t = 1, h = 0$; append

$$\mathsf{AX}(0,0,0)\ \mathsf{SHIFT}^{10}.$$

After nine Shifts, equation (2) puts the saved 1 in h, and the tenth Shift halts. This supplies an internal-PA refutation-to-new-word map without relying on the old final Shift alone. It must be applied to the prefix before that old halt: simply appending the postlude after the entire old word can discard the only exposed 1.

8.5 A weaker MP option

If only the zero push is omitted and successful MP instead returns $(b, w, b, 0)$, the same initial DROP and PUSH in the MP compiler clause provide the missing zero. Keep the old MP suffix after the two premise pushes: two Shifts, MP(b), AX$(b, 0, 0)$ and ten Shifts. It then returns $(b, w, 0, 0)$ or halts early. The stronger branch (1) has the shorter proof-word suffix above and can also shorten the register program by replacing the preserving copy with a destructive transfer into h.

9 The reverse implication inside Peano arithmetic

The upstream formal theorem is

$$\mathrm{PA} \vdash \neg H_{196} \longrightarrow \mathrm{Con(PA)}.$$

Its separate Lean proof of standard-model nonhalting does not, by itself, establish the converse implication in PA. The converse has a direct syntactic proof. The purpose of this section is to specify that proof and to isolate it from the external hereditarily finite model used upstream. The additional compiler described below has not been implemented or checked in Lean in this package.

9.1 A syntactic interpretation into PA

Use a conservative definitional extension of ordinary PA containing exponentiation and binary digits. Write $\beta(s, x)$ for the assertion that the xth binary digit of s is one. Two fixed PA theorems are

$$\beta(s,x) \longrightarrow x < s, \qquad \neg\beta(x,x).$$

The extension can be eliminated by primitive recursive proof translation; its consistency predicate is therefore interchangeable with the ordinary one for this argument.

Translate a decoded command formula into an arithmetic formula as follows:

$$\top^\tau = (0 = 0), \quad (x_a\ \mathrm{M}\ x_b)^\tau = \beta(x_a, x_b), \quad (x_a\ \mathrm{E}\ x_b)^\tau = (x_a = x_b),$$

and commute with implication and universal quantification. Bound auxiliary variables in the definition of β are renamed away from the surrounding binders; choose their labels above the largest label of the finite input. This is a primitive recursive, capture-avoiding substitution operation. The translation introduces no additional free variables. It is a map on syntax, not a satisfaction predicate. The decoder is primitive recursive: it recurses on strictly smaller

nonzero Cantor-coded subexpressions. Every variable label in the decoded formula of code c is less than c. Consequently x_c is fresh in that formula, a fact PA proves about the decoder.

The special code 0 requires an explicit convention. The arithmetic operation $\mathrm{I}(0,0)$ also gives 0, while the decoder reads 0 as truth. These two readings are propositionally equivalent. A proof compiler uses this fixed equivalence in the zero case; it must not assert literal syntactic equality between the two readings. Code 1 translates to $\beta(x_0, x_0)$, whose negation has a fixed PA proof.

There is a related normalization for derived connectives. Coded negation $\mathrm{N}(c) = \mathrm{I}(c,1)$ initially translates to $\tau(c) \to \beta(x_0, x_0)$, not to literal arithmetic negation. The fixed PA theorem $\forall x\, \neg\beta(x,x)$ gives a uniform proof that these are equivalent. The same normalization applies to coded conjunction and existential quantification, including inside binders. These fixed normalization proofs are part of the axiom templates below; the argument does not silently decode code 1 as a primitive falsity symbol.

9.2 Uniform finite comprehension is a proof schema

For every arithmetic formula $\varphi(x, \vec{y})$, PA proves

$$\forall n\, \exists s < 2^n\, \forall x < n\, (\beta(s,x) \leftrightarrow \varphi(x, \vec{y})). \tag{4}$$

Here is the proof, with its uniformity made explicit. Induct on n using the displayed existential formula as the induction formula. At $n = 0$ take $s = 0$. At the successor step, use excluded middle for $\varphi(n, \vec{y})$. If it is false, retain s; if it is true, replace s by $s + 2^n$. The standard binary-digit lemmas show that the lower digits are unchanged and that the new digit has the required value. The size bound becomes $s' < 2^{n+1}$.

This describes a primitive recursive map from the code of φ to a PA proof of (4). It inserts φ into one full-induction instance and fixed propositional and arithmetic proof templates. PA proves that this map produces valid proofs by checking the templates and the induction-axiom recognizer. No assertion of the truth of arbitrary formulas, and no uniform reflection principle, is used.

Take

$$\psi(x) = (x = b \vee \beta(a,x)) \wedge \varphi(x) \quad \text{and} \quad n = a + b + 1.$$

All possible members of a, as well as the distinguished point b, are below n. Formula (4) yields a number s with

$$\forall x\, \big(\beta(s,x) \leftrightarrow ((x = b \vee \beta(a,x)) \wedge \varphi(x))\big).$$

In particular,

$$\forall x\, ((\beta(a,x) \to \varphi(x) \to \beta(s,x)) \wedge (\beta(s,x) \to \varphi(x))),$$
$$\varphi(b) \to \beta(s,b).$$

This is the pointed-separation statement needed by the gate.

For the actual gate, a, b, c are variable labels, and the matrix is the arithmetic translation of the formula with code c, viewed as a formula in the variable x_b. The guard $c \neq a, b$ and the decoder freshness lemma ensure that the set variable x_c is fresh. One must distinguish the free distinguished-point occurrence of x_b from its bound occurrence inside the separation clause; rename the latter to a fresh variable before inserting the comprehension proof. If $a \neq b$, the preceding construction applies with x_a as the fixed container parameter. If $a = b$, the membership antecedent under the quantifier becomes $\beta(x,x)$ and is false. In that case the simpler choice $s = 2^{x_b}$ when $\varphi(x_b)$ holds, and $s = 0$ otherwise, proves both the point clause

and $\forall x(\beta(s,x) \to \varphi(x))$. This separate case avoids treating the captured occurrence of x_a as a fixed parameter. Equality and membership vacuity are ordinary quantifier theorems under the same guard. Classical propositional reasoning combines these facts into the translation of the exact bundled gate.

All six remaining axiom heads translate to fixed logical or equality theorem schemas. The restricted propositional head translates to a classical tautology; its numerical restriction does not affect soundness. The translation respects modus ponens and generalization. Thus there is a primitive recursive proof compiler J for which PA proves

$$\mathrm{Der}_{196}(d,c) \longrightarrow \mathrm{Prf}_{\mathrm{PA}}(J(d), \tau(c)). \tag{5}$$

When $c = 1$, compose with the fixed proof of $\neg\beta(x_0, x_0)$ to obtain a PA refutation. This is the precise syntactic replacement for an appeal to external semantic soundness.

More explicitly, define J by recursion on the annotated proof tree. At an axiom leaf insert its fixed logical/equality proof template, or the comprehension template just constructed for a gate. At an MP node append the translated premise proofs and one MP step, inserting the fixed zero-code equivalence when necessary. At a generalization node append the corresponding generalization. The usual finite-proof list concatenation and reindexing operations are primitive recursive. An induction on the coded proof tree verifies the output proof predicate. This defines the mathematical compiler; writing its concrete Gödel-code implementation and verifying that implementation in Lean remain separate tasks.

9.3 Command safety under consistency

For a command store (t, w, d, h), retain proof codes for the translations of t, h, and every entry of the finite stack encoded by w. The initial store has zeros everywhere, so fixed proofs of truth supply the invariant. Stack underflow produces code 0 and is harmless. Each command preserves the invariant by primitive recursive operations on proof codes:

- An axiom command appends proofs supplied by the head and gate compilers and by generalization of the existing proof of h.
- A successful modus-ponens test applies modus ponens to the existing proofs; a failed test writes zeros, for which the fixed truth proof is used.
- A shift only moves existing proofs and stack entries. Its halt test is $t + h = 1$, so one of the proved formula codes would be 1. Under Con(PA), this is impossible by (5) and the fixed refutation of its translation.

This is an arithmetical invariant involving proof codes. Induction over a coded command computation is an instance of full PA induction. It works for internal, possibly nonstandard command computations as well as for ordinary finite computations.

9.4 From commands to the literal tape machine

The final bridge is the two-level positive-continuation argument already used by the upstream external simulation. Each safe command boundary has a finite positive register-program continuation to the next safe command boundary. Each register instruction has a finite positive tape continuation to its next frame. The arithmetic versions of the pairing, unpairing, register and tape-macro contracts are total in PA. Their loop proofs use decreasing natural-number counters; the finite per-site coverage checks select the applicable contract.

In particular, the existing arithmetic modules `OriginalModelCoverage` and `Endpoint InternalCodedSourcePhysicalStep` supply all-model instruction contracts and positive coded physical continuations. `InternalCodedSourceDispatchTotal` supplies positive coded command continuations. Their conclusions are stronger than external standard-model simulation alone.

Project the auxiliary machine traces to the literal 196-state machine by relabeling their control vectors. The upstream theorem `StateMapTrace.step_maps` proves the pointwise projection for every nonpadding control. The same finite table check preserves that property at each next step. All physical root frames have the fixed nonpadding control 81, so induction along any positive frame continuation proves the required projection, with unchanged length and tape vectors. The packaged theorem `StateMapProjection.segment_to_literal` is specialized to blank-start halting segments; the positive-segment version just described is an additional composition obligation, not a claim that this specialized theorem already proves nonhalting. Apply the principle below to the projected continuations. An independent backwards simulation is not needed.

For clarity, the elementary logical principle behind this bridge is the following. Suppose I is an arithmetical configuration invariant, the initial configuration satisfies I, and every I-configuration has a positive finite execution to another I-configuration. A deterministic machine cannot have a finite halting run from an I-configuration. Prove this inside PA by strong induction on the proposed halting length n. For a continuation of length $k > 0$, if $k \leq n$, the next invariant configuration would halt in $n - k < n$ steps. If $k > n$, the continuation would extend strictly beyond the first halt, contradicting the definition of the transition relation. The induction predicate may contain I; full PA induction permits that predicate.

Applied to the proof-code invariant, the startup trace and the two levels of macro simulation, this gives the mathematical implication

$$\mathrm{PA} \vdash \mathrm{Con(PA)} \longrightarrow \neg H_{196}.$$

Together with the upstream arithmetized refutation-to-halting compiler it gives the desired biconditional. Its exact implementation status must remain visible: the upstream Lean library checks the latter compiler and external nonhalting; it does not contain the syntactic interpretation compiler J or the foregoing PA-internal proof-code safety theorem. A Lean certification of the full biconditional requires adding those modules and the positive segment and invariant compositions just described. No such certification is supplied here.

9.5 What ConciseIPRA contributes

The native scalar calculi in ConciseIPRA are proved equivalent to iPRA, whose induction is quantifier-free. They do not by that fact furnish a refutation calculus for full PA. In particular the uniform finite comprehension proof above uses the full induction instance containing the arbitrary translated formula φ. Replacing the present calculus by an iPRA calculus without supplying the missing strength would change the consistency problem. Useful transferable ideas are operational: deriving equality rules from other rules, choosing antecedent order for shared instruction trees, and using a literal syntactic feature to select a rule mode. Those ideas are applicable only after the full-PA proof translations are re-established.

10 Reduction searches and the meaning of their failures

The searches targeted changes that could be justified by small proofs: local state folds, head-offset identifications, alternative AX grammars, source specializations and register layouts.

A positive table would still require the source, startup and consistency bridges. A negative bounded search establishes only that no candidate passed that search's predicate.

10.1 Finite-window local folds

The local checker computes an overapproximation of reachable configurations in a window $[-k, k]$ around the head. A move introduces either value at the newly exposed boundary cell. Starting from the blank configuration makes the abstraction sound for all actual blank-run windows. The replacement checker then refuses any unmodeled exterior read and requires exact equality of tape, head and projected state at the endpoint.

Searches included pairs with an identical branch, pairs whose branch actions have short equivalent continuations, changes to both rows, single-state bypasses, and head offsets of two cells. The finite-window constraints were tried at radii four and five in the recorded runs, with explicit local step limits. No verified one-state saving was found for the baseline or the result-in-H variant. The more elaborate same-count helper strategy from the 208 packets was also tested on 8,061,300 two-disjoint-pair/add-one-helper proposals and yielded no survivor in its 35-step local scope.

These results do not exhaust all possible tape invariants. The successful 208 reduction itself illustrates why a previously unexamined conditional context can matter. In particular, failure of this finite-window procedure does not prove that two roles cannot be identified by a stronger invariant or a different simulation relation.

10.2 Grammar, coding, and physical-price searches

The main AX macro word has length 201 and a 68-node binary DAG. Rotations and reparsing can alter the number of distinct subtrees and the availability of predecessor-bit folds. A raw grammar count is not the full objective: a layout must also preserve the resolved source control-flow graph, the maximum zero-run constraint of the return scanner, every arithmetic entry into a shared decoder, and the blank initializer.

One recorded priced search performed 59,686 rotations and priced 46,294 distinct trees in 240 seconds; its best physical count remained 196. Separate catalog searches retained hundreds of equal-count layouts. The no-long-zero-run catalog retained 129 layouts priced at 196 among 18,922 priced layouts. Additional local-fold searches on the recorded catalogs, including 68 compatible layouts with the new MP branch, found no accepted 195-state table.

Other finite searches permuted axiom emission order, changed compatible antecedent order, changed constructor orientation and selected tag assignments, and varied source register placement. The coding experiments included sixteen combinations of transposing constructors and exchanging the two quantifier/equality residues; no improvement over the baseline 68-node AX grammar was obtained in that family. Register transpositions require their own startup argument, because a permutation of logical registers changes the physical role played by the initializer.

A restricted dictionary-segmentation mixed-integer model was also run for 180 seconds. It recovered a 68-node grammar and left a relaxation gap. The numerical solver bound applies only to its finite dictionary model; it is neither a bound on all grammars nor a bound on Turing-machine state count. It is not used as a theorem in this report.

10.3 Scanner and axiom-removal proposals

Deleting one stage from the seven-zero left scanner fails concrete source frames, including site 0 with path `000000` and site 25 with path `100000`. In one site-0 example the original reaches its correct successor in 486 steps; the shortened scanner instead produces a spurious root at head offset +8 after 236 steps. Bypassing dense state 2 by state 3 also fails the primitive frame contracts.

Dropping the final source continuation is harmless for an abstract cyclic driver, but it moves the final increment into a right-child leaf position. The current hardware routes its occurrences as left-child leaves. That source-level observation therefore does not yet provide a physical state deletion.

No redundancy proof was found for the quantifier-commutation head. The obvious derivation through full specialization is circular in this particular calculus: compound vacuity for a quantified formula uses the commutation head, and the required existential elimination then depends on that vacuity. This is an unresolved optimization, not a proof that the head is indispensable. Likewise, ConciseIPRA's quantifier-free induction does not by itself preserve the full-PA consistency calibration.

11 Reproduction, artifact map, and remaining obligation

The top-level quick replay uses only Python 3 and the standard library:

```
python3 code/verify.py
```

It checks table structure and hashes, the nine-table graph census, eighteen finite helper equivalences, exact 196-state startup equality, the one-row mutations, source counts, command regression cases, the concrete primitive-frame cases, and the rejected layout's first halt. It also runs the included upstream publication/table checks. Its output is a scope-labeled JSON receipt. It does not invoke or replace Lean.

To rebuild this report:

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The main archive paths are:

Path	Contents
`report.tex`, `report.pdf`	This manuscript and its compiled PDF.
`machines/`	Published 209/196 tables and the two labeled experimental 196-state variants.
`rejected/`	The two bad Shift layouts, deliberately excluded from the retained candidates.
`code/verify.py`	Portable quick replay and a precise description of its scope.
`sources/bb196/`	Frozen upstream source and existing license notices; no new proof is silently substituted.
`sources/history209/`	Historical source excerpts, command variants, grammar and register experiments.
`sources/incoming208/`	Complete input census, selected original proof material, nine main tables, and the helper replay.
`sources/theory_audit/`	Command and PA-converse manuscripts, command model and scanner counterexamples.
`research/`	Exploratory scripts, bounded search receipts and failed-frame evidence.
`SHA256SUMS`	Hashes of the delivered files.

The full upstream formal check is the documented command `bash scripts/check.sh -full` inside `sources/bb196`, with its specified Lean toolchain and dependencies. It was not run here. Some optional search scripts use NumPy/SciPy or a C++ compiler; their recorded results remain explicitly bounded. Repeating them is not part of the standard-library quick replay.

The remaining task is concrete: obtain a literal table with at most 195 working states, prove its source/runtime and blank-start contracts, and compose the arithmetized proof translations for that exact table. The current artifact stops before that task is achieved. The two reduced command interpreters and the comparison data may serve as starting points, but no negative experiment here proves that the requested construction cannot exist.

References

[1] James Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, source, paper and Lean development. BusyBeaver, pinned PA196 snapshot.

[2] James Chang, *Nonhalting and PA-independence for a 209-state Turing machine*, source, paper and Lean development. BusyBeaver, pinned PA209 snapshot.

[3] ProveIt Contributors, incoming consistency-machine construction and research packets. ProveIt, pinned incoming directory. Individual archive identities are in the included manifest.

[4] ProveIt Contributors, *ConciseIPRA*. ProveIt, pinned ConciseIPRA directory.